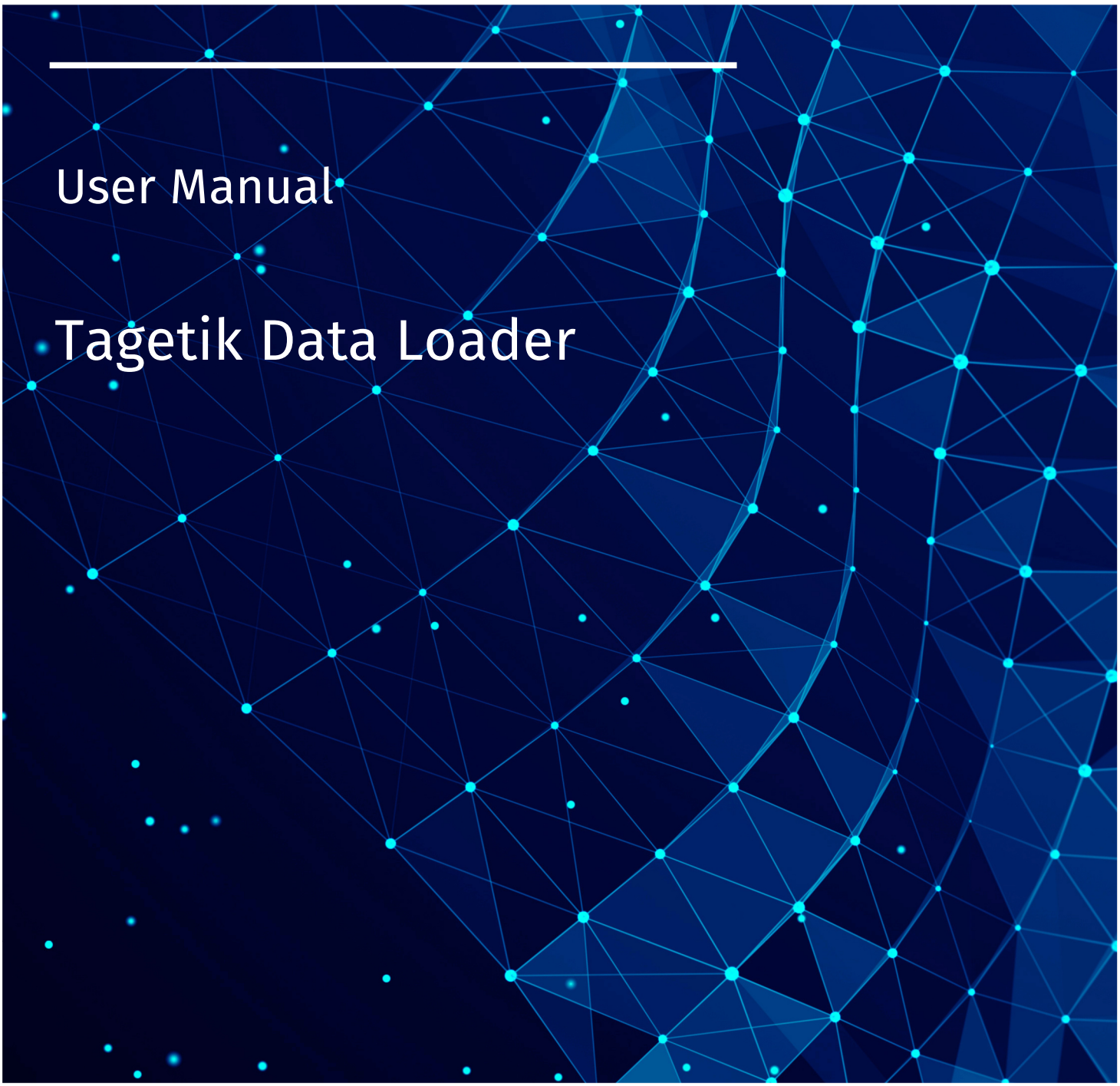


User Manual

Tagetik Data Loader

This manual provides an overview of the features of CCH Tagetik Data Loader.

Version: 5.3.22

Wolters Kluwer

wolterskluwer.com

Latest update: 30 May, 2017

Contents

Tagetik Data Loader - Introduction **5**

Purpose of this manual 5

Contents 5

Tagetik Data Loader - overview **6**

Tagetik Data Loader - architecture **8**

Framework layer 8

Services layer 9

Application layer 9

Tagetik Data Loader - installation and configuration **10**

Installation 10

Configuration 10

Java Home configuration 10

Definition of other Java and Karaf settings 11

Karaf configuration definition 11

General setting and connection to CCH Tagetik 11

Authentication configuration 13

Log configuration 13

Datasource configuration 14

Path configuration 17

Security and encryption of sensitive data 17

Starting Karaf 17

Starting as an operational system service 17

Tagetik Data Loader - Registration and monitoring on CCH Tagetik **19**

Registration of a new TDL 20

TDL management 20

TDL update 21

Configuration on CCH Tagetik 21

Endpoint **22**

URL 22

Tagetik Application 24

Agent 25

Database 26

File 28

OData service 29

Tagetik Data Loader - Introduction

Purpose of this manual

This manual provides an overview of the functionality of the Tagetik Data Loader, a client that enables the transfer of data to and from CCH Tagetik, from high cloud or on-premises systems.

Contents

Below is a summary of the topics covered in this manual.

Section	Content
Summary	Provides an overview of the Tagetik Data Loader's main functionalities, data transfer modes and possible configurable scenarios
Architecture	Describes the Tagetik Data Loader architecture
installation and configuration of Tagetik Data Loader	Describes how to install and configure Tagetik Data Loader
Registration and monitoring on CCH Tagetik	Describes how to register, monitor and manage Tagetik Data Loader on CCH Tagetik.
Endpoint configuration on CCH Tagetik	Describes how to configure an Endpoint on CCH Tagetik so that data integration functional modules (ETL, QDL, ...) can transfer data to and from CCH Tagetik using the Tagetik Data Loader

Tagetik Data Loader - overview

Tagetik Data Loader (hereinafter "TDL") is a client that allows data to be transferred from/to CCH Tagetik to/from other systems. Data can be displayed as files of database records.

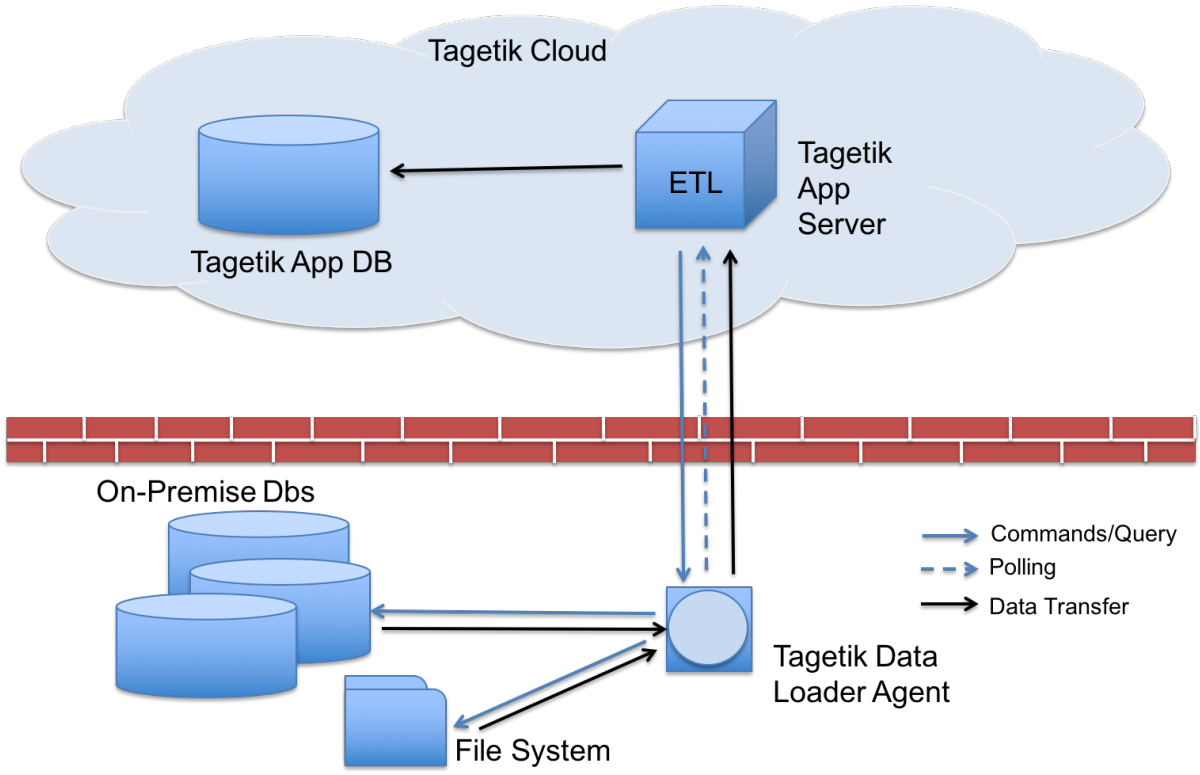
TDL is designed to be installed on-premise on the customer's network where the data is stored (private network or cloud) in order to enable data transfer via CCH Tagetik's ETL or QDL module (installed on-premise or on the cloud).

TDL allows you to select the desired transfer protocol (HTTP or HTTPS). In the case of secure data transfer (via HTTPS), TDL allows communication to take place without the customer having to create an exception in the company firewall.

Thus, Tagetik Data Loader can operate in four different types of environment (for each type of environment, TDL can transfer data in both protected mode (HTTPS) and unencrypted mode (HTTP)):

- Tagetik on-premise, TDL on-premise;
- Tagetik on-premise, TDL on cloud;
- Tagetik Cloud, TDL on-premise;
- Tagetik Cloud, TDL on cloud.

The following diagram refers to case 3.



The installation and configuration of Tagetik Data Loader involves:

- the **installation and configuration of Tagetik Data Loader**. Install the Tagetik Data Loader, configure it (connection to CCH Tagetik, logs), set up database connections, configure file transfer paths.
- **registration and monitoring of Tagetik Data Loader on CCH Tagetik**. Register, monitor and manage Tagetik Data Loader on CCH Tagetik.
- the **configuration of Endpoints on CCH Tagetik**. Enable data integration functional modules (ETL, QDL, ...) to transfer data to and from CCH Tagetik using the Tagetik Data Loader.

Tagetik Data Loader - architecture

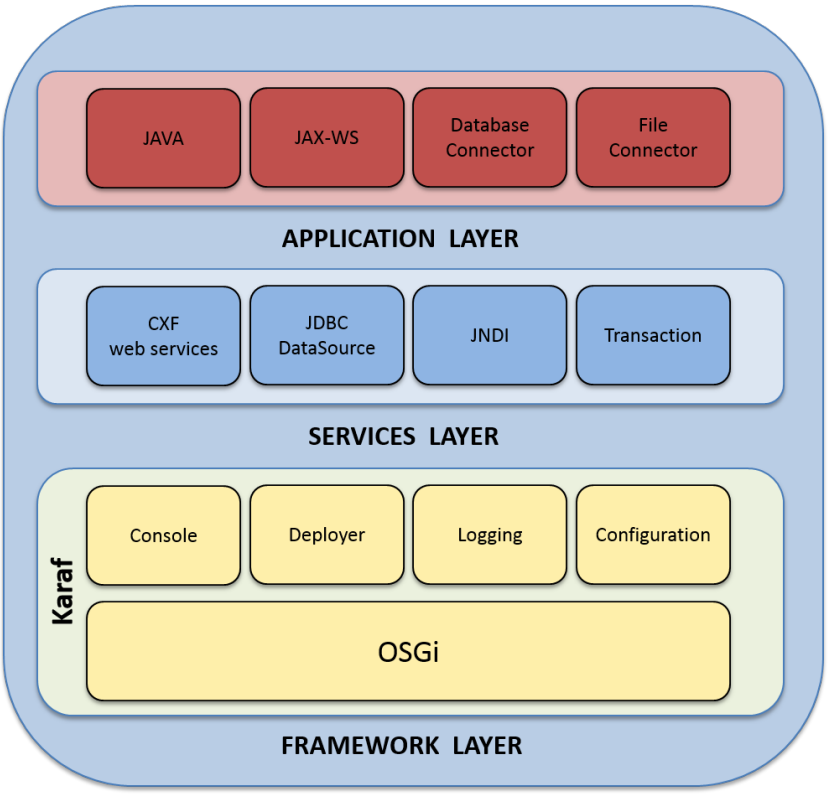
Tagetik Data Loader (hereinafter "TDL") is a programme written in Java installed and run on-premises by the customer and is able to execute tasks driven by CCH Tagetik.

TDL is structured in modules, is 'lightweight' and is dynamic, i.e. it allows modules to be installed, started, stopped and removed at runtime, thus without the need for reboots

These functionalities are provided by the use of an OSGi (Open Services Gateway initiative) framework. TDL is in fact based on Apache Karaf 4.4.0, a small OSGi framework that provides a lightweight container in which to deploy the various components and applications. Karaf 4.4.0 supports the latest OSGi R8 specification.

The TDL architecture consists of three layers as shown below:

- the first layer is the framework
- the second is the service layer
- finally, the third is the application layer



Framework layer

Apache Karaf is a small OSGi framework, which provides a 'lightweight' container in which the various components and applications can be deployed. The framework supports the following functionalities:

- **'hot update'**. Karaf supports hot deployment of OSGi bundles by tracking jar files within the [home] / deploy directory. Whenever a jar file is copied into this folder, it is installed without requiring a restart of the container
- **dynamic configuration.** The configuration of services is done using properties files within a dedicated directory. These configurations are monitored and changes to properties files are propagated to the services
- **log system.** Karaf supports a large range of logging frameworks (slf4j, log4j, etc.) that can be configured via a single file
- **administration console**. It provides an Unix-like console through which the container can be fully managed. It is also possible to extend the console and create your own customised commands.

Services layer

The service layer, located above the framework layer, interacts with the application layer to communicate with customer-developed applications that need to access and use these services. The available services are:

- **JDBC DataSource Factory**. Provides services to access databases from OSGi applications running in Karaf and to extract data from the database by installing DataSources as OSGi services
- **JNDI**. Karaf Naming (JNDI) is a feature that provides a centralised register/query function for OSGi services. JNDI API is typically used to access services from a system directory

Application layer

The Application layer is where the applications developed by Tagetik reside. It consists of java programmes that implement connectors, execute tasks and communicate with Tagetik via web services.

Tagetik Data Loader - installation and configuration

Installation

To install the Tagetik Data Loader, simply **unzip** the file **TagetikDataLoader** <version> **_ ApacheKaraf***<version>***.zip** into a directory (e.g. *<custom_path>/TagetikDataLoader/*).

Inside are two directories:

- **apache-karaf-***<version>*: contains the Apache Karaf container, the Tagetik Data Loader jars and the default JDBC drivers. The directory can be renamed to **apache-karaf**.
- **tgk-configuration-***<version>*: contains the externalisation of the Karaf configuration (/etc) in which the configuration files of the Tagetik Data Loader are also located.

Configuration

Tagetik Data Loader (TDL) requires the configuration of a set of parameters necessary for it to function correctly and fully:

- Java Home configuration
- definition of the other Karaf settings (memory, etc.)
- definition of Karaf configuration
- general configuration and configuration of the connection to CCH Tagetik
- authentication configuration
- logs configuration
- configuration of datasources for the databases access
- configuration of paths for accessing file system directories

Java Home configuration

In order to configure Java Home, it is necessary to access the *setenv* file*(setenv.bat* if installation is carried out on Windows operating systems) located in *<Karaf home>/bin* and define JAVA_HOME.

 The JDK version to be used is version 11.

E.g. linux: *export JAVA_HOME="/usr/lib/jvm/openjdk-11.0.15_10"*

E.g. windows *set JAVA_HOME=C:\Progra~1\Java\openjdk-11.0.15_10*

List of tested and supported JDKs:

- Open JDK 11.0.15
- Oracle JDK 11.0.15
- Amazon Corrected 11.0.15.9.1

- MS Open JDK 11.0.15
- IBM Semeru 11.0.15
- SapMachine 11.0.15.0.1

Definition of other Java and Karaf settings

To define the other Java and Karaf settings, it is necessary to access the *setenv* file *(setenv.bat* if installing on Windows operating systems) located in *<Karaf home>/bin* and add the options. If Karaf is run via Windows service, after running the wrapper installation command, configure the *wrapper.conf* file available inside the configuration directory. If a custom name is specified inside the wrapper installation command, the file will be *karaf-<name>.conf*

E.g. linux:

export JAVA_MIN_MEM=256M # Minimum memory for the JVM

export JAVA_MAX_MEM=1024M # Maximum memory for the JVM

export JAVA_PERM_MEM=128M # Minimum perm memory for the JVM

export JAVA_MAX_PERM_MEM=256M # Maximum memory for the JVM

E.g. windows

set JAVA_MIN_MEM=256M # Minimum memory for the JVM

set JAVA_MAX_MEM=1024M # Maximum memory for the JVM

set JAVA_PERM_MEM=128M # Minimum perm memory for the JVM

set JAVA_MAX_PERM_MEM=256M # Maximum memory for the JVM

Karaf configuration definition

It is recommended to externalise the Karaf configuration from the *<Karaf home>/etc* folder to another directory so that all customisations can be managed when updating the Karaf container. Once outsourced, it is possible to set up the configuration directory on Karaf by accessing the *setenv* file (*setenv.bat* if installing on Windows operating systems) located in *<Karaf home>/bin* and defining the KARAF_ETC.

E.g. linux: *export KARAF_ETC="/home/user/TagetikDataLoader/tgk-configuration"*

E.g. windows *set KARAF_ETC=C:|Progra~1|TagetikDataLoader|tgk-configuration*

General setting and connection to CCH Tagetik

To configure the TDL for both the connection to CCH Tagetik and to change certain parameters relating to its functionality, it is necessary to access the *tgk.properties.cfg* file in the Karaf configuration directory.

The parameters to be configured are:

- **tgk.agent.polling.interval**. Indicates at what intervals (in seconds) the TDL verifies requests for transactions to be taken over. It is recommended not to set up a value lower than 30 seconds
- **tgk.agent.tmpfolder**. Indicates the path reserved for the TDL to save temporary resources generated by packet data transmission. If it is not specified, the temporary files will be saved into the temporary directory of Java.
- **tgk.agent.write.db.uniquetransaction**. Indicates whether the writing to the database of data transferred in several packets is to be done in a single transaction at the end of the reception of the last packet (true) or whether it is to be done separately after each received packet (false)
- **tgk.agent.write.db.batchsize**. Indicates the number of statements sent simultaneously to the database to be executed at write time (default 10000)
- **tgk.agent.read.db.fetchsize**. Indicates the number of records to be retrieved simultaneously from the database when reading data (default <empty>)
- **tgk.endpoint.address**. CCH Tagetik endpoint URL (e.g. https://alpha.saastagetik.com/prod)
- **tgk.endpoint.protocol.https**. Drives the use of https (true) or http (false) protocol

If secure data transfer (HTTPS) is active, it is necessary to install the certificates (both client- and server-side) on a path visible to the TDL and then configure the following parameters:

- **tgk.server.ks.password**. Password chosen when generating the server-side certificate. It must be written following the syntax specified in Security and encryption of sensitive data
- **tgk.server.ks.path**. Complete path of the server-side certificate
- **tgk.client.ks.path**. Complete path of the client-side certificate
- **tgk.client.ks.password**. Password chosen when generating the client-side certificate. It must be written following the syntax specified in Security and encryption of sensitive data

If the communication between TDL and CCH Tagetik is managed by a proxy server, the following can also be configured:

- **tgk.proxy.server**. Indicates the IP of the proxy server
- **tgk.proxy.port**. Indicates the proxy server port
- **tgk.proxy.user**. Indicates the user accessing the proxy server
- **tgk.proxy.password**. Indicates the password with which the user defined in the previous property can access the proxy server. It must be written following the syntax specified in Security and encryption of sensitive data

Example 1 (HTTPS secure transfer)

```
tgk.agent.pollingInterval=30
tgk.agent.tmpfolder=/home/user/TagetikDataLoader/tgk-configuration/tmp/
tgk.agent.write.db.uniquetransaction=true
tgk.agent.write.db.batchsize=10000
tgk.agent.read.db.fetchsize=
tgk.endpoint.protocol.https=true
```

```
tgk.endpoint.address=https://tagetik.installation.url.com/tagetikcpm
tgk.server.ks.path=/home/TDL/certificates/server.truststore
tgk.server.ks.password=server_side_certificate_password
tgk.client.ks.path=/home/TDL/certificates/web_certificate.p12
tgk.client.ks.password=client_side_certificate_password
```

Example 2 (HTTP transfer)

```
tgk.agent.pollingInterval=30
tgk.agent.tmpfolder=/home/user/TagetikDataLoader/tgk-configuration/tmp/
tgk.agent.write.db.uniquetransaction=true
tgk.agent.write.db.batchsize=10000
tgk.agent.read.db.fetchsize=
tgk.endpoint.protocol.https=false
tgk.endpoint.address=http://tagetik.installation.url.com/tagetikcpm
```

Authentication configuration

To enable the communication between TDL and CCH Tagetik, it is necessary to define the authentication parameters. They can be found in the file *tgk.agent.cfg* in the Karaf configuration directory.

The parameters are as follows:

- **tgk.agent.id**. Unique ID associated with the TDL instance. It is automatically generated upon the first launch of Karaf.
- t**gk.agent.secret**. SecretKey to authenticate the TDL on CCH Tagetik. It must be defined after the first launch of Karaf, once the ID has been automatically generated, and must be defined using the syntax specified in Security and encryption of sensitive data

With these two parameters it is possible to register the TDL on CCH Tagetik and authenticate it upon each request.

Log configuration

The TDL log system is nothing more than a customization of the log system of Apache Karaf. The log configuration can be found in the file *org.ops4j.pax.logging.cfg* in the Karaf configuration directory and is a standard Log4j configuration file.

This file contains the following section with the specified values:

```
#Tagetik agent appender
log4j2.logger.tagetik.name = com.tagetik.cpm
log4j2.logger.tagetik.level=INFO
log4j2.logger.tagetik.additivity=false
log4j2.logger.tagetik.appenderRef.TagetikRollingFile.ref = TagetikRollingFile
```

log4j2.appender.tagetik.type = RollingRandomAccessFile

log4j2.appender.tagetik.name = TagetikRollingFile

log4j2.appender.tagetik.fileName = ${karaf.data}/log/tagetik.log

log4j2.appender.tagetik.filePattern = ${karaf.data}/log/tagetik.log.%i

log4j2.appender.tagetik.append = true

log4j2.appender.tagetik.layout.type = PatternLayout

log4j2.appender.tagetik.layout.pattern = ${log4j2.pattern}

log4j2.appender.tagetik.policies.type = Policies

log4j2.appender.tagetik.policies.size.type = SizeBasedTriggeringPolicy

log4j2.appender.tagetik.policies.size.size = 16MB

For the meaning of the various options and of the values to be assigned, refer to the Log4j specifications.

Log files are saved by default in *data/log/karaf.log*

Based on the default settings above, the TDL log file is named *tagetik.log* and is generated on a daily basis. Moreover, every day the file generated on the previous day is renamed by adding a suffix given by the production date.

Datasource configuration

In case database operations are required such as:

- reading data to be transferred to CCH Tagetik
- writing of data extracted from CCH Tagetik

TDL requires *.cfg* files to be created and configured in the Karaf configuration directory.

Here are some templates of such files:

SqlServer (driver JTDS) - **tgk.datasource-mssql-jtds.cfg**

osgi.jdbc.driver.class=net.sourceforge.jtds.jdbc.Driver

pool=dbcp2

xa=false

url=jdbc:jtds:sqlserver://<servername>:<port>/<dbname>

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

SqlServer (driver Microsoft) - **tgk.datasource-mssql-microsoft.cfg**

osgi.jdbc.driver.class=com.microsoft.sqlserver.jdbc.SQLServerDriver

pool=dbcp2

xa=false

url=jdbc:jtds:sqlserver://<servername>:<port>;databaseName=<dbname>

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

Oracle - **tgk.datasource-oracle.cfg**

osgi.jdbc.driver.class=oracle.jdbc.OracleDriver

pool=dbcp2

xa=false

url=jdbc:oracle:thin:<serverName>:<port>:<serviceName>

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

PostgreSQL - **tgk.datasource-postgresql.cfg**

osgi.jdbc.driver.class=org.postgresql.Driver

pool=dbcp2

xa=false

url=jdbc:postgresql://<serverName>:<port>/<dbName>

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

SAP Hana - **tgk.datasource-saphana.cfg**

osgi.jdbc.driver.class=com.sap.db.jdbc.Driver

pool=dbcp2

xa=false

url=jdbc:sap://<servername>:<port>/?databaseName=<dbname>

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

IBM DB2 - **tgk.datasource-ibmdb2.cfg**

osgi.jdbc.driver.class=com.ibm.db2.jcc.DB2Driver

pool=dbcp2

xa=false

url-
=jd-
bc:d-
b2://<serverName>:<port>/<dbName>:retrieveMessagesFromServerOnGetMessage=true;

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

Netsuite - **tgk.datasource-netsuite.cfg**

osgi.jdbc.driver.class=com.netsuite.jdbc.openaccess.OpenAccessDriver

pool=dbcp2

xa=false

url-
=jd-
bc:n-
s://<servername>:<port>;ServerDataSource=NetSuite.com;encrypted=1;CustomProperties=
(AccountID=<accountId>;RoleID=<roleId>)

user=<username>

password={TGK_ENCRYPT}<userPassword>{TGK_ENCRYPT}

dataSourceName=<symbolicName>

vendor=tagetik

 The datasources passwords must be entered (both upon definition and edit) with the syntax specified in Security and encryption of sentitive data

Adding a custom datasource

From the Karaf command line via the command *service:list DataSourceFactory* it is possible to display the installed JDBC drivers.

To add a new JDBC driver it is necessary to:

- copy the **[driver_name].jar** into the *<Karaf home>/lib/endorsed* folder
- run the *install -s wrap:file:./lib/endorsed/[driver_name].jar* command from the Karaf command line

- add a *.cfg* file relating to the new datasource into the Karaf configuration directory (like those described above)
- restart Karaf

 The simultaneous presence of two JDBC drivers from the same vendor causes problems at runtime. If it is necessary to install a new JDBC driver (E.g. : a later version), the previous one has to be uninstalled.

Path configuration

In case operations are needed on files such as:

- file transfer to CCH Tagetik
- writing files produced by CCH Tagetik

TDL requires the *tgk.repository.cfg* file to be created and configured in the Karaf configuration directory.

Inside, a mapping list between the logical path name and the real absolute path can be configured.

The syntax is typical of the *.properties files, thus:

<path-1-logical-name>=<path-1-value>

<path-2-logical-name>=<path-2-value>

Security and encryption of sensitive data

For keystore and datasource passwords, there is a mechanism for encrypting the values.

To define the value of the password, the following syntax as to be used: *<password_property_name>={TGK_ENCRYPT}<password_value_to_encrypt>{TGK_ENCRYPT}*

After saving the configuration file, Tagetik Data Loader will automatically encrypt the value of the property and replace it in the same configuration file.

Starting Karaf

Once installation and configuration have been carried out, Tagetik Data Loader can be used by starting Karaf via the *karaf* command*(karaf.bat* if installation is carried out on Windows operating systems) present in *<Karaf home>/bin*.

Starting as an operational system service

It is also possible to start Karaf as an OS service by executing the command *wrapper:install --name "TDL" --display "TDL" --description "TDL"* from the Karaf command line and afterwards:

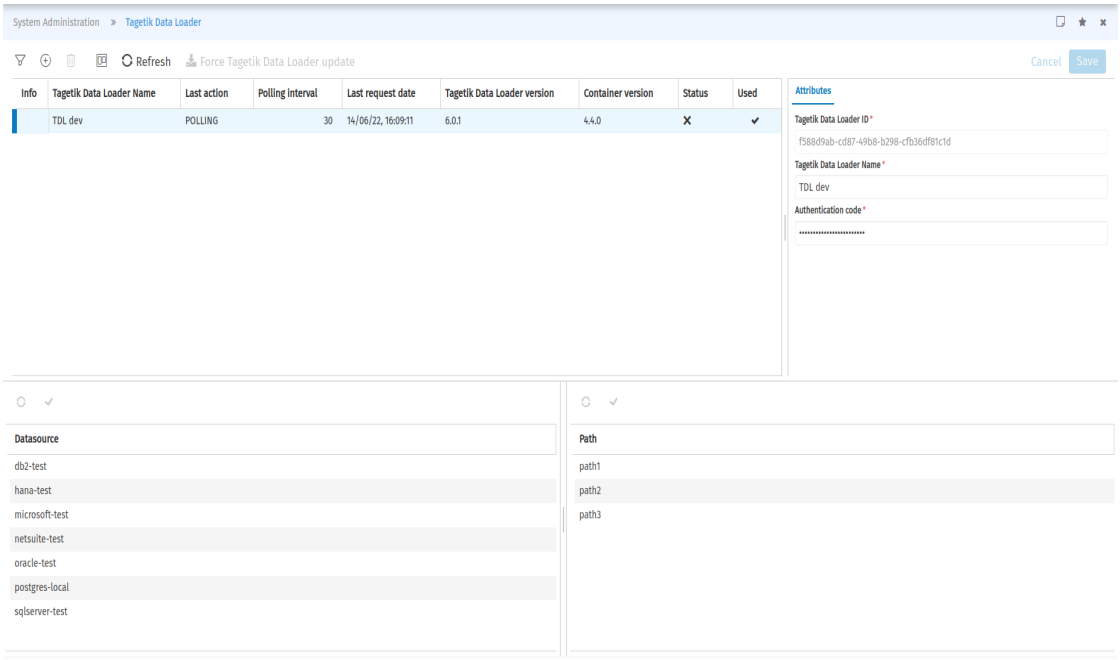
- for Windows: run from the command prompt the command *TDL-service.bat install* (or *karaf-service.bat install*), present in the *<Karaf home>/bin* directory
- for Linux: follow the instructions shown once you run the *wrapper:install* command .

Tagetik Data Loader - Registration and monitoring on CCH Tagetik

CCH Tagetik provides the admin users with a window to:

- register new TDLs so that CCH Tagetik enables communication with them
- remove TDLs no longer active and not used in any parametrization (Endpoint)
- display the information and the status of the configured TDLs in real time
- display the last or current TDL operation performed
- request the manual update of a TDL if the automatic update fails
- request the updating of the list of datasources configured on the TDL and their verification
- request the updating of the list of paths configured on the TDL and their verification

This monitor functionality is accessible from the tile of the same name available on the Repository home page.



The configured TDLs are displayed at the top of the window, and for each one is shown:

- **Tagetik Data Loader Name**: TDL description defined by the user
- **Last action**: last command executed by the TDL or in progress
- **Polling interval**: specifies at what intervals (in seconds) the TDL checks for operations to be taken over
- **Last requested date**: date of the last 'heart beat' made by the TDL
- **Tagetik Data Loader version**: TDL version installed on Karaf
- **Container version**: Karaf container version
- **Status**: displays the status of the TDL via an indicator:

- ✓ : on-line
- ✗ : off-line
- ⚠ : on-line but locked
  - Authentication failed
  - Waiting for automatic update
  - Automatic update in progress
  - Automatic update failed
  - Manual upgrade required
- **Used**: indicates whether the TDL is used in one of the configured Endpoints.

There are two different sections at the bottom of the window:

- **Datasource**: where the system displays the Datasources' logical names,
- **Path:** where the system displays the logical names of Paths

both configured on Karaf for the current TDL. For both sections there are the following buttons:

- ↻ : allows updating data in the section by retrieving the information from the configuration files of the current TDL. This button is only enabled if the selected TDL is online
- ✓ : performs a check in order to check the validity of the selected Datasource/Path. The result of the verification, whether positive or negative, is notified at the end of the operation by an alert containing an appropriate message.

Registration of a new TDL

CCH Tagetik only enables communication with a TDL if it has been previously registered.

A new TDL can be registered via the button ⊕. The system enables the *Attributes* panel where you must specify:

- **Tagetik Data Loader ID**: unique identifier of the TDL. The field must be filled with the ID automatically generated by Karaf when the TDL is first started in the *tgk.agent.cfg* file in the Karaf configuration directory.
- **Tagetik Data Loader Name**: TDL description defined by the user
- **Authentication** code: identifies the authentication code associated with the TDL; it must match the one specified in the *tgk.agent.cfg* file in the Karaf configuration directory.

TDL management

Once a TDL has been registered, you can:

- edit the **Tagetik Data Loader Name** and **Authentication Code** via management in detail,
- refresh the displayed information (e.g. **Status**) via the button ↻ Refresh

- remove the configuration of a TDL via the  button.

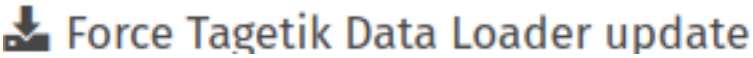
Editing and deletion are only possible if the TDL is not used in a parametrization of an Endpoint.

TDL update

The version of a TDL (e.g. 6.0.1) consists of three pieces of information:

- *Major*: used to identify 'breaking-change' such as changing the Karaf container or changing the communication protocol,
- *Minor*: used to identify the addition of new features that do not compromise the use of existing ones,
- *Patch*: used to identify corrections to existing functionalities.

How it is updated depends on the difference between the version of the reference TDL on CCH Tagetik and the version of the TDL on Karaf :

- If the two *majors* are different, the **Status of** the Tagetik Data Loader will be changed to  (warning) and *TO_UPGRADE* will be displayed as **Last Action**. This means that a manual update of the Karaf/TDL container will have to be carried out.
- if the *Minor.Path* of the TDL on CCH Tagetik is smaller than that of the TDL on Karaf, no update is necessary.
- if the *Minor.Path* of the TDL on CCH Tagetik is greater than that of the TDL on Karaf, an automatic update will be performed. In the **Last Action** field the commands *TO_UPDATE*, *UPDATING* and, if the update was successful, the POLLING command (which certifies that communication is active again) will be performed, otherwise the *UPDATE_FAILED* command will be displayed. In this case, a manual update can be retried via the 

button.

Configuration on CCH Tagetik

In the configuration of the CCH Tagetik Repository, there are settings relating to the use of TDL:

Data write: data packet size (Kbytes): Indicates the maximum size in KBytes into which data extracted from CCH Tagetik to be transmitted to Tagetik Data Loader must be divided and sent.

Read Data : Data Packet Size (Kbytes): Indicates the maximum size in KBytes into which data extracted from Tagetik Data Loader is to be divided and sent to CCH Tagetik.

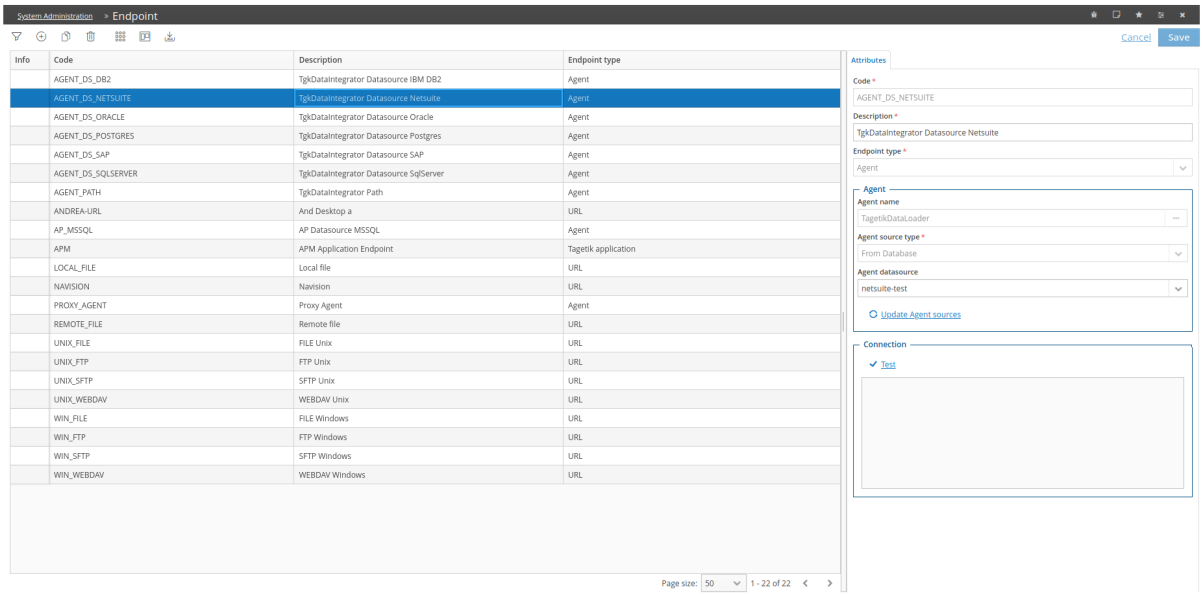
Time out (min): Indicates the minutes CCH Tagetik waits for the Tagetik Data Loader to respond to a command. After this time has passed, a 'Timeout' error is returned by the application. The default time is 30 minutes.

Endpoint

The access to external resources can be set up by defining an Endpoint.

The Endpoints can be used when reading/writing from an ETL Job and other modules of Tagetik.

The Endpoint configuration can be accessed from the administrator user web interface via the *Endpoint* tile on the Repository home page.

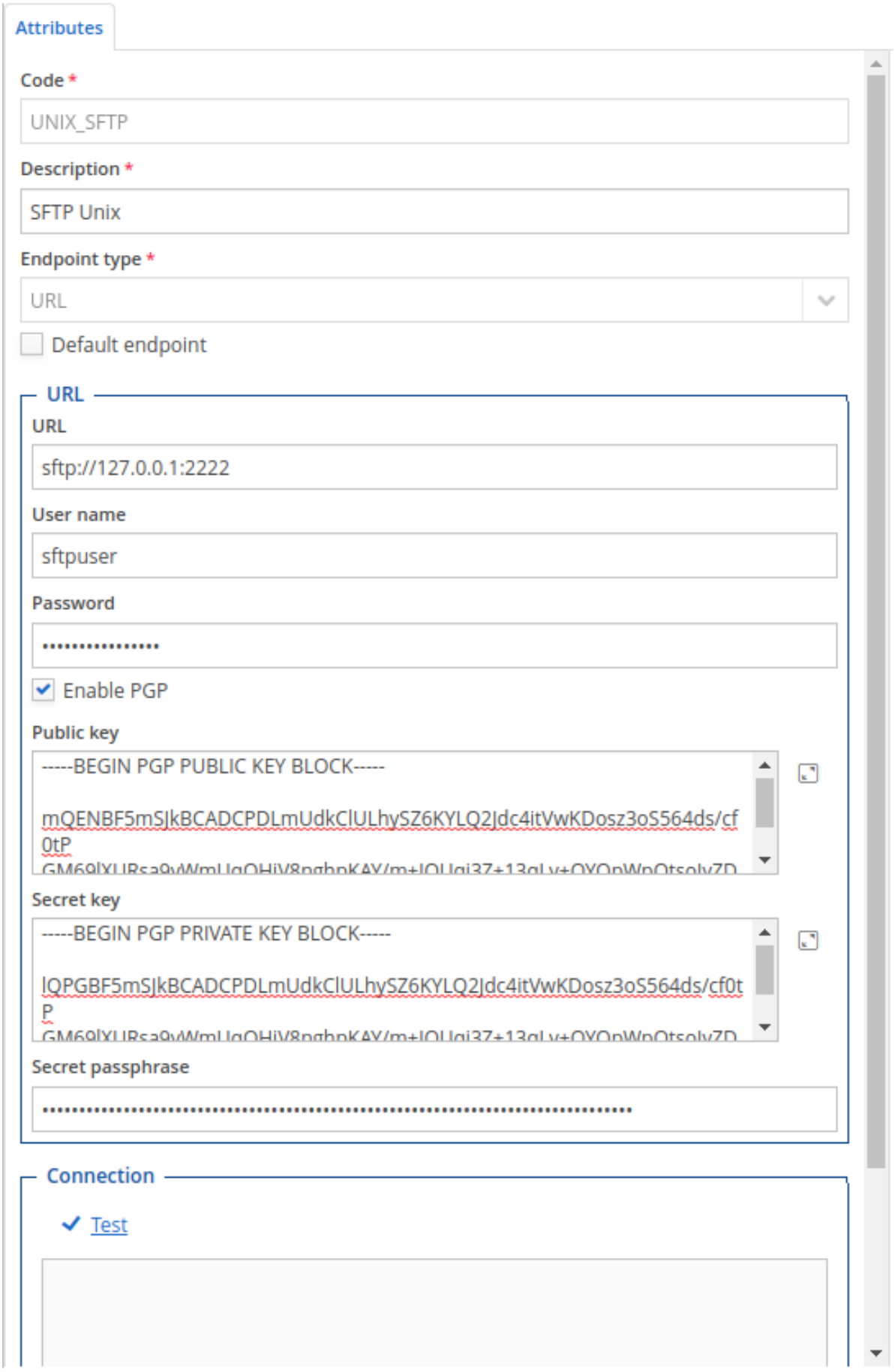


First of all, it is necessary to define:

- **Code**. Unique ID by which the Tagetik's modules refer to the endpoint.
- **Description**. Extended description.
- **Endpoint type**. Access type that can be *URL*, *Agent or Tagetik Application*, *OData V2 service* and *OData V4 service*.
- **Default endpoint**: it is visible only for the URL Endpoints. It indicates that the files produced by CCH Tagetik tools (such as application Import/Export) that require an extended storage have to be saved in the remote URL set up in the endpoint. If this option is not active for any Endpoint, then the files produced b the CCH Tagetik tools will be stored in the directory setup as *writable folder*.

URL

If the selected type is **URL**, it is possible to refer to a resource by using a protocol, based on the syntax specified in the **URL** field.


Attributes
Code *
UNIX_SFTP
Description *
SFTP Unix
Endpoint type *
URL
Default endpoint
URL
URL
sftp://127.0.0.1:2222
User name
sftpuser
Password
Enable PGP
Public key
-----BEGIN PGP PUBLIC KEY BLOCK-----
mQENBF5mSJkBCADCPDLmUdkClULhySZ6KYLQ2Jdc4itVwKDosz3oS564ds/cf
0tP
Secret key
-----BEGIN PGP PRIVATE KEY BLOCK-----
lQPGBF5mSJkBCADCPDLmUdkClULhySZ6KYLQ2Jdc4itVwKDosz3oS564ds/cf0t
P
Secret passphrase
Connection
Test

Available protocols:

Protocol	URL Windows example	URL Unix example
File	file://localhost/c|/WINDOWS/ file:///c|/WINDOWS/ file://localhost/c:/WINDOWS/	file://localhost/etc/ file:///etc/
Ftp	ftp://host[:port]/url-path	
Ftp (secure)	ftps://host[:port]/url-path	
Http	http://webdav/files	
Https	https://webdav/files	
Sftp	sftp://host[:port]/url-path	

If a protocol requires the authentication, user and password must be specified in the following fields:

- **Username**. It identifies the user who can access the resource.
- **Password**. Password by which the user can access the resource (the value of the field is hidden).

Pretty Good Privacy Cryptography (PGP)

The **PGP** cryptography (option **enable PGP**) can be enabled for the **URL** Endpoint. When this option is enabled, files are decrypted when they are read and encrypted when they are written, by using the PGP standard.

Required information:

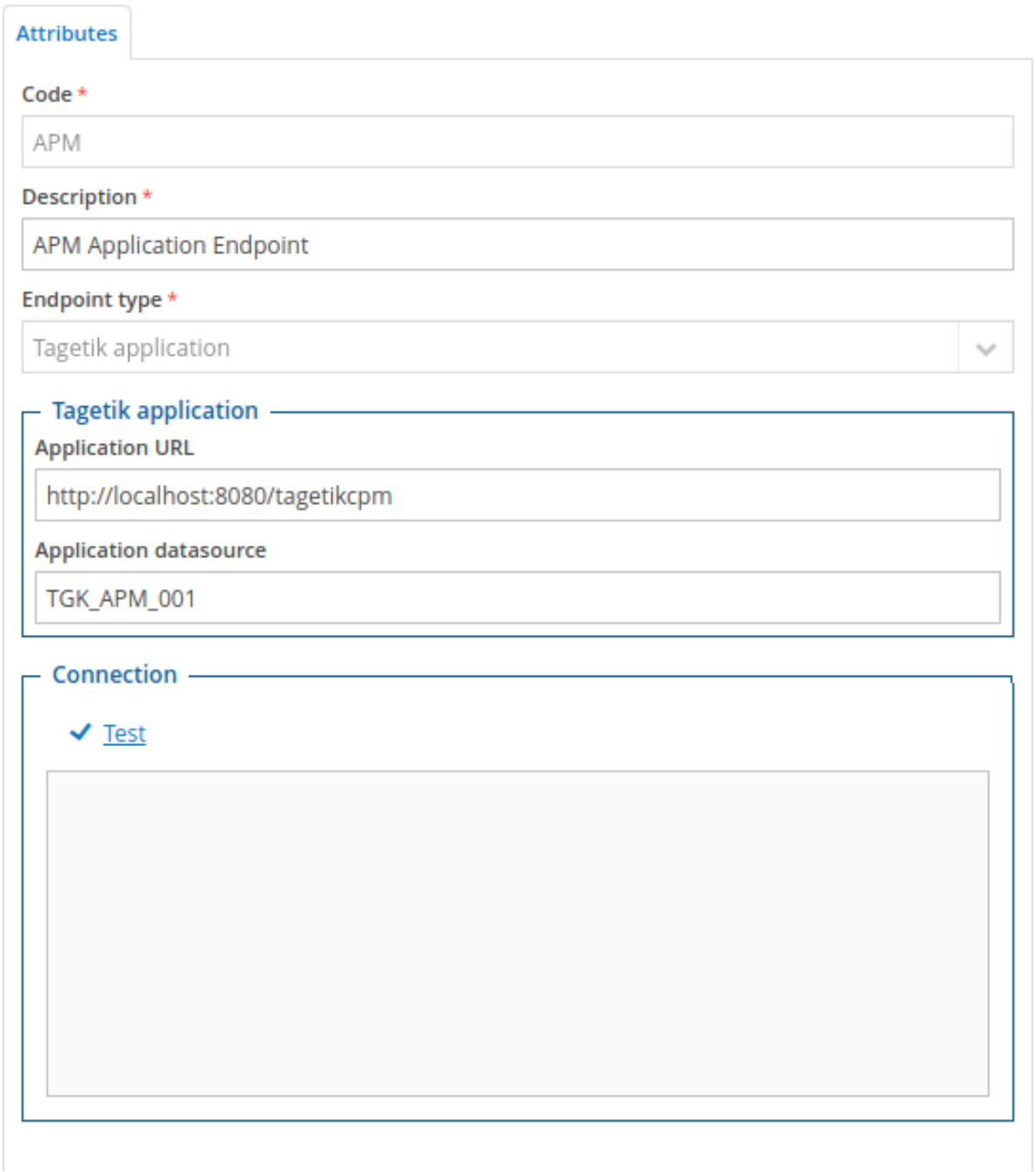
- **Public key**: the public key of the pair of keys generated by the customer
- **Secret key**: the private key of the pair of keys generated by Tagetik
- **Secret passphrase**: passphrase used to generate the pair of keys of CCH Tagetik (the value of the field is hidden).

Tagetik Application

By selecting the **Tagetik Application** type, it is possible to link a Tagetik application to a specific datasource

The configuration includes:

- **URL**. Url of a Tagetik application.
- **Datasource**. ID of one of the datasources defined for the Tagetik application



Agent

If the selected type is Agent, it is possible to refer to a Tagetik Data Loader Agent previously defined on Tagetik via the Agent Monitor tile.

The configuration includes:

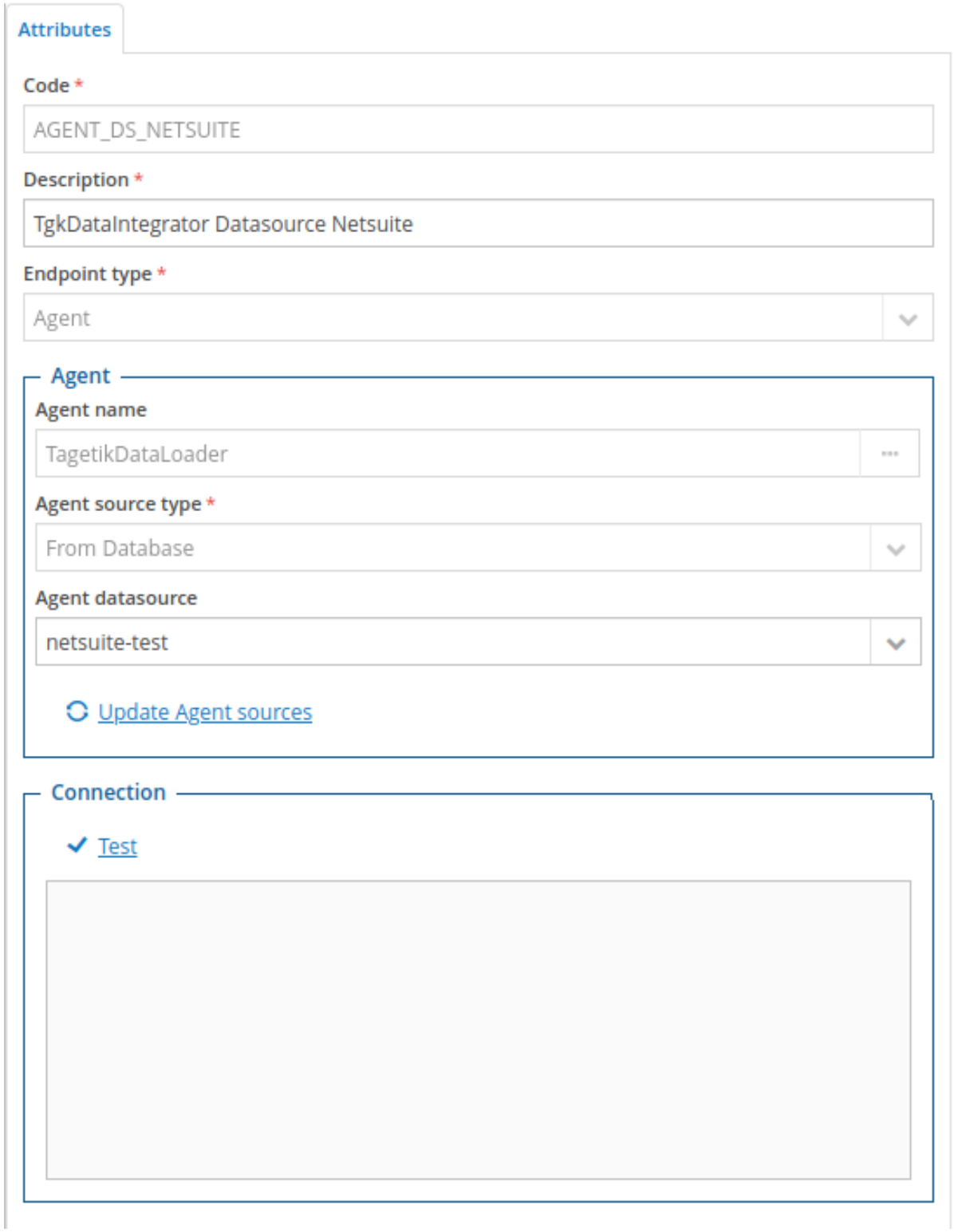
- **Agent name**: to be chosen among the defined ones.

- **Agent source type**: Source type from which to extract / load data; it can be either *From database* or *From file*

After choosing the source type from which to extract / load data, by clicking on **Update Agent sources** the system updates the list of datasources or paths configured on the selected Tagetik Data Loader Agent.

Database

If the selected type is **From database**, it means that the resource to be used is the databases configured on the Tagetik Data Loader Agent.

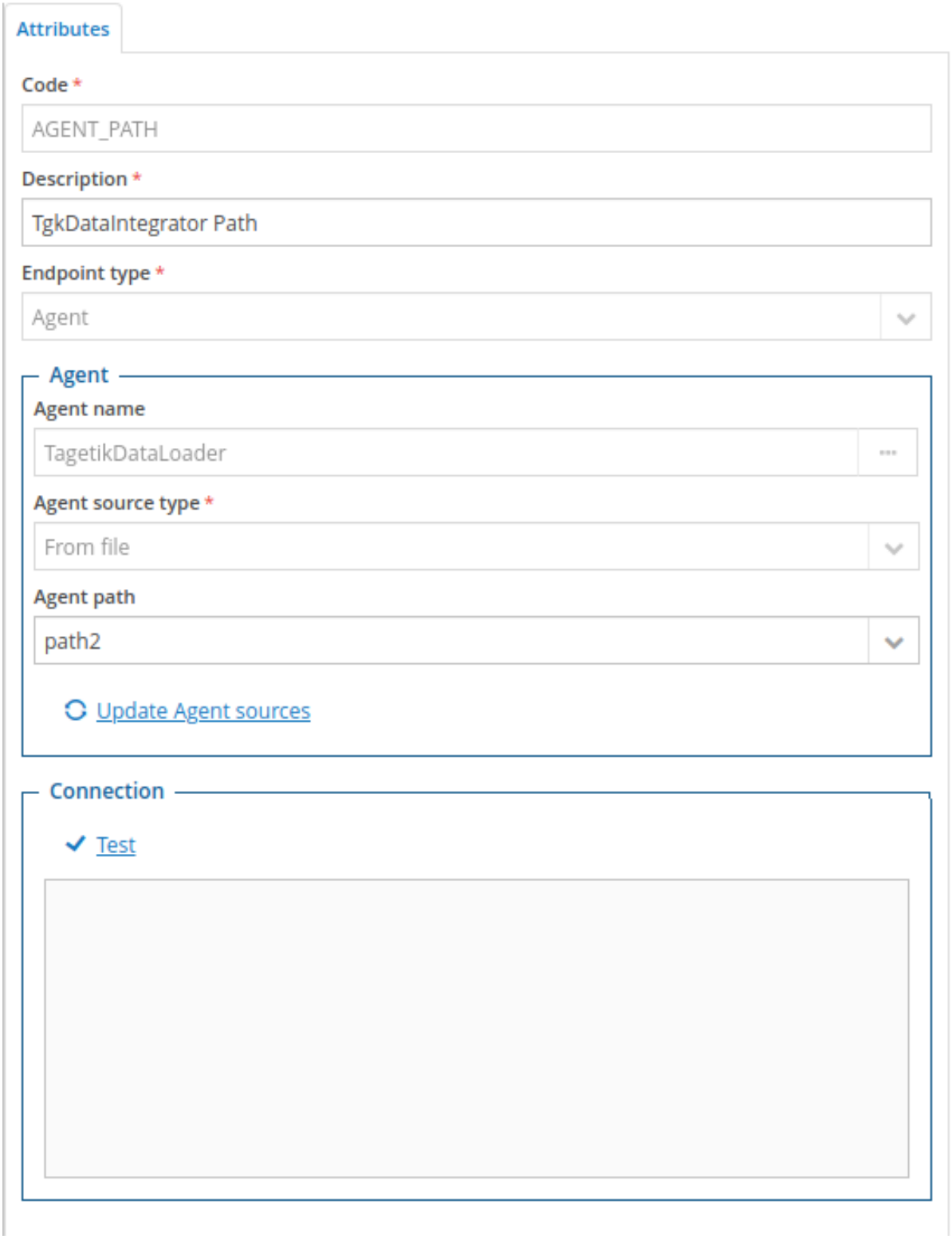


The configuration includes:

- **Agent datasource**: Name of the datasource from which to extract / load data.

File

If the selected type is **From File**, it means that the resources to be used are the repositories configured on the Tagetik Data Loader Agent.



The configuration includes:

- **Agent path**:Name of the path on which the application can run the read/write operations.

OData service

By selecting **OData V4 service** or **OData V2 service**, it is possible to leverage an Odata service to read a collection of enrtities (the EntitySet described in the Entity Data Model of the service's root).

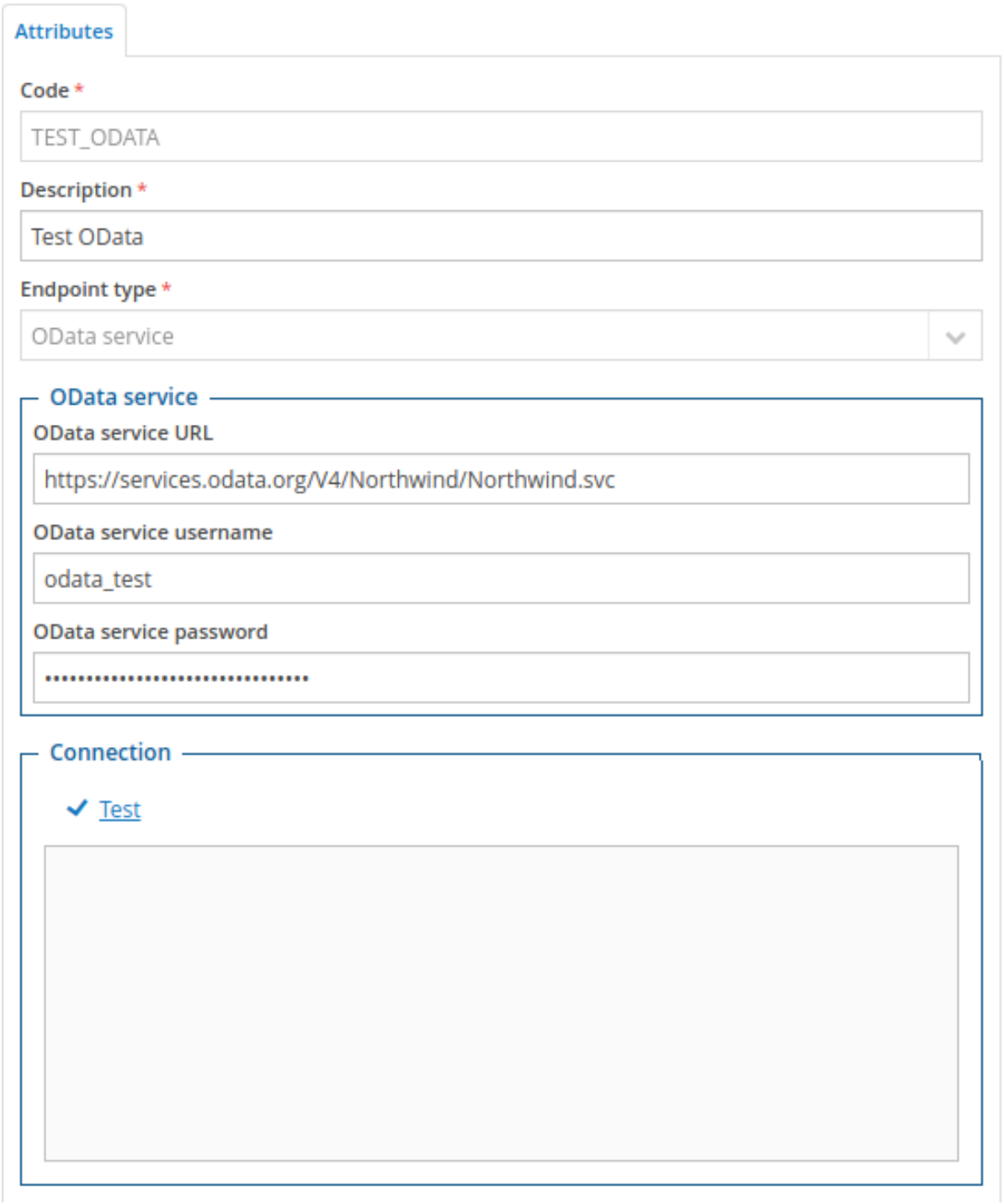
The configuration requires the following information:

- **OData service URL**: the URL of the root of the Odata service.

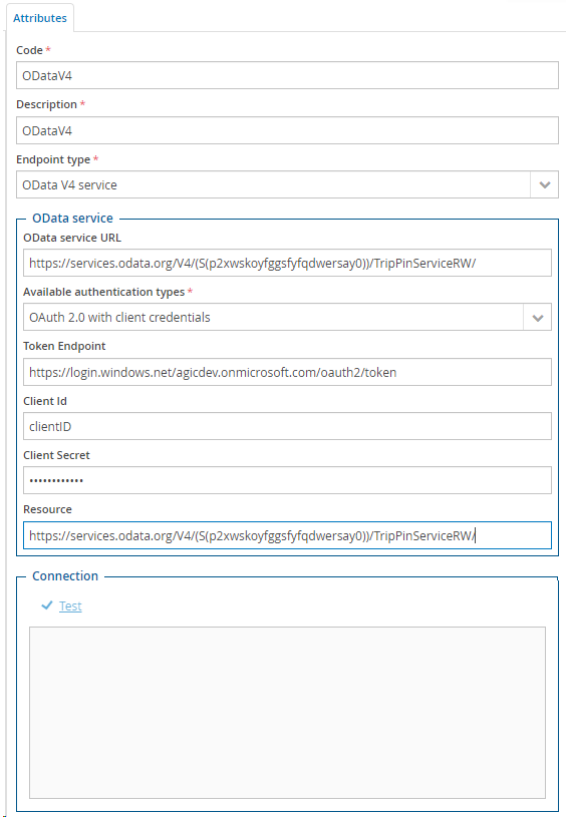
  If the service requires an authentication, then it is possible to choose one among the **Available authentication types**.

Currently the available authentication typologies are:

- **Basic Authentication**:
  - **OData service username**: username enabled to get the access
  - **OData service password**: the password linked to the username defined above



- **Oauth 2 with client credentials:**
  - **Token Endpoint**: URI to contact the IDP to get the token
  - **Client Id**: the ID of the client application that can access the source
  - **Client Secret**: the password associated to the Client Id (see above)
  - **Resource**: in case of OData v4 endpoint and Dynamics365 IDP then this option identifies the source you are getting the access on



For each Endpoint, it is possible to verify the validity of the inserted information and test the connection to the remote resource just by clicking on the **Test** button available in the *Connection* panel. Once the test is completed, a message containing the detailed result of the test pops up.